AGENCYOS

Development & AI Coding Agent System

Document 13 of the AgencyOS Master Documentation Set

DEVELOPER AGENTS • CODING WORKFLOW • TASK ORCHESTRATION • GIT • BUILDS • TESTING • RETRIES • SELF-VERIFICATION • HANDOFFS

Purpose: Define a reliable autonomous software-development workforce that can take approved scope and UI, plan implementation, write code, test it, verify its own work, recover from failures, and hand off validated artifacts without falsely declaring incomplete work complete.

1. Core Principle

> Developer agents are autonomous executors inside a controlled engineering system. They do not get to redefine scope, bypass quality gates, or declare project completion solely from their own output.

- Approved scope is the product boundary.
- Approved UI/prototype is the design baseline.
- PM Agent orchestrates work.
- Specialist developer agents implement assigned tasks.
- Git provides versioned source history.
- Automated tests and QA provide independent verification.
- Builds must be reproducible enough to validate the submitted change.
- Self-verification is mandatory before handoff.
- Critical production/deployment actions remain policy-gated.
- Every task and handoff must leave machine-readable evidence.

2. End-to-End Development Lifecycle

> APPROVED SCOPE/UI → TECHNICAL PLAN → TASK BREAKDOWN → REPOSITORY/BRANCH → IMPLEMENT → LOCAL VALIDATION → TEST → BUILD → SELF-VERIFY → CODE REVIEW/QA → PM ACCEPTANCE → HANDOFF → DEPLOYMENT GATE

- A task can return to implementation when verification fails.
- A failed build or test never counts as successful completion.
- Repeated failures escalate rather than looping indefinitely.

3. Development Inputs

- Active scope version.
- Approved UI version.
- Prototype version where applicable.
- Feature requirements.
- Acceptance criteria.
- Technical constraints.
- Existing architecture.
- Repository state.
- Coding standards.
- Security policies.
- Environment configuration.
- Dependency policies.
- Database/API contracts.
- Test requirements.
- Deployment constraints.

Developer agents should receive the minimum context required for their task plus links to authoritative project artifacts.

4. Developer Agent Workforce

Recommended specialist roles

• Frontend Developer Agent.

• Backend/API Developer Agent.

• Database/Data Agent.

• Mobile Developer Agent.

• Integration Agent.

• DevOps/Build Agent.

• Security/Code Review Agent.

• Test Automation Agent.

• Bug Fix Agent.

• Refactoring/Performance Agent.

• Documentation Agent.

Not every project needs every agent. The Orchestrator creates the required workforce from project scope.

5. Developer Agent Ownership

• Each task has one primary executing agent.

• Supporting agents may be assigned explicitly.

• The PM Agent owns coordination.

• The Orchestrator owns routing.

• QA owns independent validation.

• No two agents should unknowingly edit the same file/task concurrently.

• Ownership and handoffs are recorded.

6. Technical Planning

Before coding

• Read active scope.

• Read approved UI/prototype.

• Inspect repository architecture.

• Identify affected modules.

• Identify dependencies.

• Identify API/data contracts.

• Identify risks.

• Break feature into implementation tasks.

• Define acceptance criteria.

• Define test plan.

• Define rollback/recovery considerations.

> The agent should plan before editing. A short plan is preferable to uncontrolled repository-wide changes.

7. Task Model

Task fields

• Task ID.

- Project ID.
- Milestone ID.
- Feature ID.
- Requirement ID.
- Scope version.
- UI version where applicable.
- Task type.
- Description.
- Acceptance criteria.
- Dependencies.
- Assigned agent.
- Priority.
- Branch/worktree.
- Status.
- Attempt number.
- Files changed.
- Tests.
- Build result.
- Verification result.
- Blocker.
- Handoff state.

8. Task States

> BACKLOG → READY → ASSIGNED → IN_PROGRESS → SELF_VERIFY → TESTING → REVIEW → ACCEPTED

- Failure states may include BLOCKED, RETRYING, REJECTED and FAILED.
- A task cannot become ACCEPTED merely because code was written.
- Acceptance requires configured evidence.
- Task state transitions are backend-controlled.

9. Task Orchestration

> PM → ORCHESTRATOR → DEPENDENCY GRAPH → AGENT ASSIGNMENT → EXECUTION → RESULT → VERIFICATION → NEXT TASK

- Build a dependency graph from the scope.
- Prioritize tasks whose dependencies are satisfied.
- Run independent tasks in parallel when safe.
- Serialize conflicting work.
- Respect resource limits.
- Track task attempts.
- Detect stalled tasks.
- Escalate blocked dependencies.

10. Parallel Development

• Parallelize independent frontend/backend/documentation work.

• Avoid parallel edits to the same high-conflict files.

• Use isolated branches/worktrees where supported.

• Merge only after validation.

• Re-run relevant tests after integration.

• Detect merge conflicts automatically.

• Do not resolve complex conflicts by blindly choosing one side.

The Orchestrator should optimize for safe parallelism, not maximum concurrency.

11. Git Repository Strategy

• One authoritative repository per project or configured repository set.

• Protected main/production branch.

• Feature/task branches or isolated worktrees.

• Meaningful commits.

• Commit messages reference task IDs.

• Pull/merge request or equivalent review record where configured.

• Tags/releases for significant builds.

• No direct production-branch edits by autonomous agents unless explicitly permitted.

• Repository access is least-privilege.

12. Branch Lifecycle

> TASK READY → CREATE BRANCH → IMPLEMENT → COMMIT → TEST → REVIEW → MERGE → INTEGRATION TEST

• Branch names should include project/task identifiers.

• Abandoned branches should be identifiable.

• Merged work is linked to the task.

• A failed branch can be retried without losing history.

• A new attempt should preserve the previous failure evidence.

13. Commit Policy

• Commit focused changes.

• Avoid unrelated refactors.

• Reference task/feature ID.

• Do not commit secrets.

• Do not commit generated artifacts unless explicitly required.

• Keep repository clean.

• Commit after a coherent validated unit of work.

Agents should not produce giant opaque commits containing unrelated changes.

14. Codebase Understanding

• Inspect directory structure.

• Read relevant architecture docs.

• Identify entry points.

- Identify shared components.
- Identify existing patterns.
- Search for related implementations before creating duplicates.
- Check existing tests.
- Check dependency versions.
- Understand configuration/environment boundaries.

> The agent should modify the existing architecture intentionally, not rebuild working pieces because it failed to find them.

15. Coding Standards

- Project-specific style guide.
- Naming conventions.
- Component/service patterns.
- Error handling.
- Logging.
- Type safety where applicable.
- Input validation.
- Security requirements.
- Testing conventions.
- Documentation conventions.
- Dependency policy.
- Performance expectations.

16. Secure Coding

- Validate untrusted input.
- Use parameterized database operations.
- Protect authentication/authorization.
- Avoid exposing secrets.
- Avoid logging credentials/tokens.
- Use secure dependency versions.
- Review permissions.
- Protect file/network access.
- Apply rate limiting where applicable.
- Perform security checks before production handoff.

> Security failures are blocking failures when policy classifies them as critical.

17. Dependency Management

- Record dependency changes.
- Prefer project-approved versions.
- Check compatibility.
- Run dependency/security checks where configured.
- Avoid unnecessary new dependencies.

• Record reason for major dependency additions.

• Lock or pin versions according to project policy.

• Rebuild after dependency changes.

18. Environment Management

• Development environment.

• Test/CI environment.

• Staging/preview where applicable.

• Production environment.

• Environment-specific configuration.

• Secret references, never raw secrets in code.

• Environment parity checks.

• Build metadata.

Developer agents should not copy production credentials into development code or commits.

19. Coding Workflow

> READ → PLAN → EDIT → FORMAT/LINT → UNIT TEST → INTEGRATION TEST → BUILD → SELF-VERIFY → HANDOFF

• Read the relevant code before editing.

• Make the smallest coherent change.

• Run formatting/linting.

• Run targeted tests.

• Run broader tests as appropriate.

• Build the affected target.

• Inspect output/errors.

• Review the diff.

• Confirm acceptance criteria.

• Prepare structured handoff.

20. Test Strategy

Test layers

• Static analysis.

• Formatting/linting.

• Unit tests.

• Component tests.

• Integration tests.

• API/contract tests.

• Database tests.

• End-to-end tests.

• UI automation.

• Build verification.

• Security checks.

• Performance checks where required.

The test matrix is project-dependent; the system should know which checks are mandatory for each task type.

21. Test Selection

• Run targeted tests for the changed module first.

• Run dependent integration tests.

• Run full relevant suite before milestone acceptance.

• Run regression suite for high-impact changes.

• Re-run tests after conflict resolution.

• Record exact commands/results.

• Do not report tests as passed if they were skipped.

22. Build System

• Build commands are project-configured.

• Build environment is recorded.

• Build version/commit is recorded.

• Build artifacts are identifiable.

• Build logs are retained where appropriate.

• Failed builds block configured transitions.

• Successful compilation alone does not prove functional correctness.

• Prototype and production builds must remain distinguishable.

23. Self-Verification

> Before saying 'completed', the developer agent must prove the task against its acceptance criteria.

• Inspect git diff.

• Check changed files.

• Confirm no accidental unrelated changes.

• Run required tests.

• Run required build.

• Check expected behavior.

• Check edge cases.

• Check error handling.

• Check security implications.

• Confirm documentation/config updates where required.

• Confirm task-to-scope mapping.

• Generate structured evidence.

> The agent's completion message is a summary of evidence, not the evidence itself.

24. Definition of Done

• Acceptance criteria satisfied.

• Implementation complete.

• Relevant tests pass.

• Required build succeeds.
• Required static/security checks pass.
• Git diff reviewed.
• No known blocking issue remains.
• Scope mapping exists.
• Artifacts are stored/linked.
• Documentation updated where required.
• Self-verification complete.
• Task handoff package generated.

25. Failure Handling

• Capture exact error.
• Classify failure.
• Check whether failure is code, environment, dependency, test, infrastructure or requirement related.
• Attempt a safe fix.
• Re-run the failed check.
• Record attempt number.
• Do not repeatedly make random changes.
• Escalate when retry budget is exhausted.

> Retries should be evidence-driven, not 'try again with different code' loops.

26. Retry Policy

Configurable controls

• Maximum attempts per task.
• Maximum autonomous edit cycles.
• Maximum time budget.
• Retry delay where appropriate.
• Allowed failure categories.
• Escalation threshold.
• Fallback agent/model.
• Fallback strategy.
• Whether a fresh context is allowed.

A retry should normally use the previous failure evidence so the agent does not repeat the same mistake.

27. Recovery Strategy

> FAILURE → DIAGNOSE → TARGETED FIX → TARGETED TEST → FULL RELEVANT TEST → BUILD → VERIFY

• Environment failure → repair environment or escalate.
• Dependency failure → inspect compatibility/lockfile.
• Test failure → inspect regression and expected behavior.
• Merge conflict → resolve with repository context, then retest.
• Requirement ambiguity → PM clarification.

• Architecture conflict → senior/reviewer escalation.

• Security finding → Security/PM/Admin gate.

• Repeated failure → human escalation.

28. Self-Healing Boundaries

• Agents may fix their own code within assigned task scope.

• Agents may fix test failures caused by their change.

• Agents may perform small refactors necessary for the task.

• Agents may not expand product scope.

• Agents may not alter business policy.

• Agents may not weaken tests just to make them pass.

• Agents may not disable security controls to bypass errors.

• Agents may not delete failing tests without authorized reason.

• Agents may not silently modify production data.

29. Test Integrity

> Tests are verification instruments, not obstacles to be defeated.

• Do not remove meaningful assertions to obtain green builds.

• Do not mock away the behavior under test without justification.

• Do not mark tests skipped without recording why.

• Do not change expected behavior without scope/requirement evidence.

• Test changes should be reviewed like production code.

30. Code Review Agent

• Inspect diff.

• Check correctness.

• Check architecture consistency.

• Check security.

• Check performance risks.

• Check maintainability.

• Check test adequacy.

• Check scope alignment.

• Check accidental secrets.

• Return findings with severity.

> The review agent should be independent enough to challenge the implementation agent.

31. QA Handoff

> DEVELOPER → STRUCTURED HANDOFF → QA → TEST → DEFECTS → DEVELOPER → RETEST → ACCEPTANCE

Handoff package

• Task ID.

• Scope/feature reference.

• Commit/branch.

• Build ID.

• Changed areas.

• Implemented behavior.

• Tests executed.

• Test results.

• Known limitations.

• Environment assumptions.

• How to verify.

• Open risks.

• Related UI/prototype version.

32. Defect Loop

• QA creates defect.

• Defect links to task/build.

• Developer receives reproducible steps.

• Developer fixes defect.

• Developer self-verifies.

• QA retests.

• Regression tests run where required.

• Defect closes only after verification.

• Repeated defects can lower project/agent health score.

A defect cannot be closed by the same agent simply because it claims to have fixed it if independent QA is required.

33. Handoff Between Developer Agents

• Frontend → Backend: API contract/dependency details.

• Backend → Frontend: endpoint/schema/auth behavior.

• Developer → QA: build and test instructions.

• UI → Developer: approved UI/design system.

• Developer → DevOps: build/deployment requirements.

• Developer → Documentation: implementation/config changes.

• Each handoff references artifacts and exact versions.

> Agents should exchange structured handoff packages, not vague messages like 'backend done, continue.'

34. Artifact & Version Management

• Source commit.

• Branch.

• Build ID.

• Package/APK/web artifact.

• Test report.

• Coverage report where applicable.

• Logs.

• Screenshots/video where useful.

• Documentation.

• Dependency lock state.

• Deployment candidate.

Every important artifact should be traceable back to source commit and project scope.

35. Build Reproducibility

• Record source commit.

• Record dependency state.

• Record build environment/toolchain.

• Record build configuration.

• Record generated version.

• Record build timestamp.

• Store checksums/identifiers where useful.

• Separate development, preview and production artifacts.

> If a build cannot be identified, it should not be treated as a reliable release candidate.

36. Database & Migration Safety

• Migration scripts are versioned.

• Migration is tested.

• Rollback strategy is considered.

• Destructive migrations require elevated review.

• Production data changes are separately controlled.

• Seed/test data is not confused with production data.

• Schema changes trigger relevant integration tests.

Developer autonomy should be lower for irreversible production database operations.

37. API & Contract Validation

• API schema/version.

• Request validation.

• Response contract.

• Authentication/authorization.

• Error codes.

• Backward compatibility.

• Integration tests.

• Frontend/backend dependency mapping.

• Contract version linked to scope where applicable.

38. UI Development Alignment

• Use approved UI version.

• Use design tokens/components.

• Match screen inventory.

- Implement approved states.
- Preserve responsive behavior.
- Map UI components to features.
- Flag deviations.
- Do not silently redesign approved UI.

> If implementation reveals a UI problem, create a design feedback/change workflow rather than changing the approved design silently.

39. Autonomous Completion Gate

> TASK → IMPLEMENTED? → TESTS PASS? → BUILD PASS? → DIFF CLEAN? → ACCEPTANCE CRITERIA PASS? → SECURITY/QUALITY PASS? → EVIDENCE COMPLETE? → HANDOFF

- Any required NO blocks completion.
- The gate is deterministic.
- Agent-generated text cannot override a failed check.
- Admin/PM may override supported gates only through the Policy Engine.

40. Project-Level Development Completion

- All required development tasks accepted.
- All required features mapped.
- All required tests passed.
- No blocking defects.
- Build candidate generated.
- QA sign-off complete.
- Deployment gate satisfied.
- Documentation/handover artifacts ready.
- Project state can advance only through configured workflow.

41. Development Monitoring

- Tasks in progress.
- Stalled tasks.
- Retry count.
- Failure categories.
- Build success rate.
- Test success rate.
- Defect reopen rate.
- Average task duration.
- Agent productivity.
- Agent failure rate.
- Code review findings.
- Scope drift.
- Dependency failures.
- Deployment readiness.

42. Admin Controls

• Pause development automation.

• Pause one project.

• Pause one agent.

• Reassign task.

• Change model/provider routing through Orchestrator policy.

• Approve/reject high-risk code actions.

• Approve production deployment.

• Set retry limits.

• Set task budgets/timeouts.

• Configure required tests.

• View repository activity.

• View build/test logs.

• Inspect agent handoffs.

• Trigger escalation.

• Emergency kill switch.

43. Security & Repository Permissions

• Least-privilege repository access.

• Separate read/write/deploy permissions.

• Protected branches.

• Secret management outside source.

• Audit repository actions.

• Restrict production credentials.

• Limit network/file access where possible.

• Require elevated approval for destructive operations.

• Rotate/revoke credentials through Admin controls.

The coding agent should have enough permission to complete its task, not unlimited infrastructure authority.

44. AI Model Routing for Coding

• Orchestrator selects coding model/provider.

• Routing may consider coding quality, context window, cost, latency and availability.

• Use fallback model/provider on failure.

• Pin model for sensitive/reproducibility-critical tasks where required.

• Record model/provider for significant coding runs.

• Policy can block providers from sensitive code/data.

• No provider should receive secrets unnecessarily.

45. Handoff to Deployment

> ACCEPTED CODE → RELEASE CANDIDATE → QA/SECURITY → DEPLOYMENT APPROVAL → DEPLOYMENT AGENT → SMOKE TEST → RESULT → ROLLBACK IF REQUIRED

• Deployment is separate from development completion.

• Exact commit/build is referenced.

• Approval is linked to exact version.

• Rollback plan exists.

• Post-deployment validation is required.

• Failed smoke test triggers rollback/escalation according to policy.

46. Handoff to Maintenance

• Production version.

• Repository state.

• Architecture documentation.

• Environment/configuration references.

• Known issues.

• Monitoring information.

• Support notes.

• Maintenance scope.

• Client handover documentation.

• Warranty/support window.

Maintenance agents need operational context, not just source code.

47. Preventing False Completion

• Completion requires evidence.

• Task status cannot be manually inferred from chat alone.

• Tests must actually execute.

• Build must actually succeed.

• Artifacts must exist.

• Acceptance criteria must be checked.

• QA must independently verify where required.

• Unresolved blockers remain visible.

• Agent confidence cannot substitute for validation.

> This is a central AgencyOS requirement: an agent must never say 'completed' when the system can prove required work is missing.

48. Example Autonomous Task

> PM assigns 'Implement login screen and authentication API'. Frontend and backend tasks are created. Agents inspect approved UI and API requirements. Backend implements API and tests. Frontend implements screen and states. Integration tests run. Build succeeds. Code review finds a missing error state. Frontend fixes it. Tests rerun. QA validates the complete login flow. Task is accepted with commit/build/test evidence.

If authentication fails repeatedly, the system escalates rather than endlessly modifying unrelated code.

49. Acceptance Criteria

• Developer agents are specialized and assignable.

• Every coding task has structured scope/context.

- Task dependencies are tracked.
- Parallel work is supported safely.
- Git branches/worktrees are controlled.
- Commits reference tasks.
- Protected branches prevent unsafe direct changes.
- Developer workflow includes planning before editing.
- Required tests are configurable.
- Builds are validated.
- Self-verification is mandatory.
- Failures produce evidence.
- Retries are bounded.
- Retries use prior failure context.
- Agents cannot weaken tests to pass.
- Code review can challenge implementation.
- QA handoff is structured.
- Artifacts are versioned and traceable.
- Database migrations are controlled.
- UI implementation references approved UI.
- Deployment is separated from development completion.
- Admin can pause/override supported workflows.
- Model/provider routing is policy-controlled.
- Project completion requires accepted development work and QA evidence.
- Agents cannot falsely declare incomplete work complete.

50. Final Architecture

> APPROVED SCOPE/UI → PM → ORCHESTRATOR → TASK GRAPH → DEVELOPER AGENTS → GIT → TESTS → BUILD → SELF-VERIFY → CODE REVIEW → QA → ACCEPTED ARTIFACT → DEPLOYMENT GATE → RELEASE → SMOKE TEST → HANDOFF

The Development & AI Coding Agent System is the engineering execution layer of AgencyOS. Its job is not simply to generate code; it must manage the full engineering loop: understand the approved product, implement it in controlled tasks, validate it, recover from failures, preserve source history and provide evidence that the work is actually complete.

> The key outcome: AgencyOS should be able to run development with minimal human interruption while retaining hard technical gates that prevent incomplete code, failed builds, skipped tests, uncontrolled scope changes and unverified releases from being treated as completed work.